

Reviewer Pack — Minimal Local Indeterminacy in Quantum Information Geometry ...

Entry 5e57ec20e58b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 05:25:27 UTC

Minimal Local Indeterminacy in Quantum Information Geometry and Circuit Monotone Width

Entry ID: 5e57ec20e58b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 05:25:27 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Geometry **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every boolean circuit C with n gates, the minimal local indeterminacy ($\text{mli}(C)$) of its associated quantum state is linearly correlated with its monotone width $w_{\text{mon}}(C)$, such that $\text{mli}(C) = \Theta(w_{\text{mon}}(C))$.

Rationale (proposer's reasoning):

Quantum information geometry provides a framework to study quantum systems, which may reveal hidden structures in boolean circuits. Minimal local indeterminacy quantifies the uncertainty of quantum states and could potentially expose subtle properties of circuits.

Taxonomy category: Geometric_Quantum (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 82017ef455aa425a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every boolean circuit C with n gates ($n \leq 40$), if the Pearson correlation coefficient between the minimal local indeterminacy $\text{mli}(Q(C))$ and the monotone width $w_{\text{mon}}(C)$ is greater than or equal to 0.8 for at least 24 out of 30 random seeds, then support the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=1.6s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_circuit(n):
        circuit = []
        for _ in range(n):
            gate_type = random.choice(['AND', 'OR', 'NOT'])
            if gate_type == 'NOT':
                inputs = [random.randint(0, 1)]
            else:
                inputs = [random.randint(0, 1) for _ in range(random.randint(2, 3))]
            circuit.append((gate_type, inputs))
        return circuit

    def evaluate_circuit(circuit):
        stack = []
        for gate_type, inputs in reversed(circuit):
            if gate_type == 'NOT':
                result = 1 - inputs[0]
            elif gate_type == 'AND':
                result = all(inputs)
            else:
                result = any(inputs)
            stack.append(result)
        return stack.pop()

    def calculate_minimal_local_indeterminacy(state):
        # Placeholder for actual calculation
        return random.random() * len(state)

    def calculate_monotone_width(circuit):
        # Placeholder for actual calculation
        return len(circuit)
```

```

mli_values = []
w_mon_values = []

for _ in range(30):
    n = random.randint(5, 40)
    circuit = generate_boolean_circuit(n)
    state = evaluate_circuit(circuit)
    mli = calculate_minimal_local_indeterminacy(state)
    w_mon = calculate_monotone_width(circuit)

    if len(mli_values) >= 30:
        break

    mli_values.append(mli)
    w_mon_values.append(w_mon)

if len(mli_values) < 30:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": len(mli_values),
        "n_max": max(len(circuit) for circuit in [generate_boolean_circuit(n) for n in range(5, 41)]),
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

mean_mli = sum(mli_values) / len(mli_values)
mean_w_mon = sum(w_mon_values) / len(w_mon_values)
covariance = sum((mli - mean_mli) * (w_mon - mean_w_mon) for mli, w_mon in zip(mli_values, w_mon_values))
variance_mli = sum((mli - mean_mli) ** 2 for mli in mli_values) / len(mli_values)
variance_w_mon = sum((w_mon - mean_w_mon) ** 2 for w_mon in w_mon_values) / len(w_mon_values)

pearson_corr_coeff = covariance / (math.sqrt(variance_mli) * math.sqrt(variance_w_mon))

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": pearson_corr_coeff,
    "instances_tested": 30,
    "n_max": max(len(circuit) for circuit in [generate_boolean_circuit(n) for n in range(5, 41)]),
    "conjecture_holds": pearson_corr_coeff >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.2:
    print(f"RESULT: FALSIFIED counterexample=\"not_enough_support\" first_failing_seed={seeds[0]}")
else:
    print(f"RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f1d2a3d5.py", line 101, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f1d2a3d5.py", line 59, in run_trial
    mli = calculate_minimal_local_indeterminacy(state)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f1d2a3d5.py", line 46, in calculate_minimal_local_indeterminacy
    return random.random() * len(state)
           ^^^^^^^^^^^^^
TypeError: object of type 'int' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the error in the test code that caused it to crash. Once the issue is resolved, rerun the test with a valid dataset to assess the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13717
2	preregistration	ollama_remote	glm4:latest	0	0	9396
3	novelty	ollama_remote	glm4:latest	0	0	14435
4	novelty	ollama_remote	glm4:latest	0	0	15553
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18624

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17867
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19350
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	80373
9	judge	ollama_remote	glm4:latest	0	0	58288

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 247602 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/5e57ec20e58b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/5e57ec20e58b.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/5e57ec20e58b.tar.gz (if generated)